
boolean



Controlled Elements

Aka: Come lavorare con gli elementi dei form

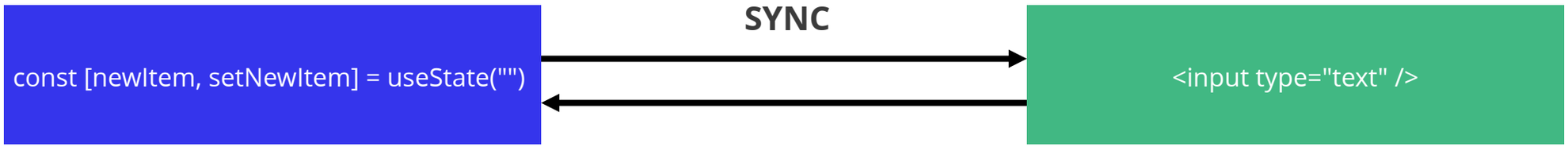
Controlled Element

Aka: Come lavorare con gli elementi di un form

Nelle applicazioni più complesse, è spesso necessario interagire con gli utenti attraverso moduli come quelli per il **login**, la **registrazione** o i **form per l'inserimento o la modifica dei dati**.

Mantenere il **valore** di questi campi **sincronizzato** con lo **stato** interno del componente è una pratica comune.

Questo approccio offre un **maggiore controllo** sull'interfaccia utente, consentendo un'interazione fluida e reattiva con l'utente: **quando lo stato cambia, il valore del campo viene aggiornato e viceversa**.



Controlled Element

Campi di form e state

Per legare il valore di un campo di un form a un dato in React, basta inserire uno stato all'interno dell'attributo `value`.

```jsx
import { useState } from "react";

export default function App() {
  const [firstName, setFirstName] = useState('Marco');

  return (
    <input type="text" value={firstName} />
  )
}
```

Il valore del campo sarà dunque legato a quello dello state.



Controlled Element

Campi di form e state

Usando i *devTools* di React, possiamo provare a cambiare il valore dello stato e vedremmo riflesso il cambiamento nel nostro input.

Tuttavia, se provassimo a scrivere direttamente nel campo per cambiarne il valore scopriremmo che questo non reagisce!

Non è tutto! Un errore in console, ci segnala che qualcosa non va!

```
▶Warning: You provided a `value` prop to a form react-dom.development.js:86
field without an `onChange` handler. This will render a read-only field. If
the field should be mutable use `defaultValue`. Otherwise, set either
`onChange` or `readOnly`.
    at input
    at App (http://localhost:5173/src/App.jsx:21:37)
```

Controlled Element

Uncontrolled element

Questo errore, ci appare perchè il nostro elemento input **è fuori dal nostro controllo**!

Il suo valore è legato allo state, ma non abbiamo modo di cambiarlo.

Si tratta quindi di un **uncontrolled element**, poichè il dato è legato in una sola direzione: dallo state al value, ma non viceversa!

Il legame che abbiamo in questo tipo di situazione viene detto **one-way data binding.**



Controlled Element

Sincronizziare i dati

Tuttavia, se leggiamo bene il nostro errore, ci viene fornita la soluzione: l'evento **onChange()**.

Possiamo intercettare l'evento scatenato sull'input quando l'utente ne cambia il valore e passare il nuovo valore allo stato, modificandolo!

Una volta modificato, lo stato aggiornerà di nuovo il value del componente, dandoci quindi una **sincronizzazione** tra i dati del nostro componente il valore dell'input!

In questo modo otterremo un **two-way** **data binding** ed il nostro diverrà un **controlled element!**



Controlled Element

Sincronizzare i dati

Usiamo quindi l'attributo `onChange` per intercettare l'evento e raccogliere il valore inserito dall'utente.

Successivamente, chiamiamo `setFirstName` per riassegnare il nuovo valore allo stato.

Una volta cambiato il valore dello stato, React aggiornerà il value dell'input!

```jsx
import { useState } from 'react'

function App() {

  const [firstName, setFirstName] = useState('Marco');

  return (
    <>
      <input
        type="text"
        value={firstName}
        onChange={e => { setFirstName(e.target.value) }}
      <p>Il tuo nome è {firstName}</p>
    </>
  )
}

export default App
```

Controlled Element

Le fasi di un controlled element

All'avvio: lo state determina il value iniziale



l'utente scrive: onChange, usa setFirstName e modifica lo state



lo state è stato modificato, quindi riaggiorna il value





LIVE CODING

Controlled Element

Evento onSubmit

Oltre a gestire un singolo input, è spesso necessario gestire un tag form.

Pur avendo un singolo campo, inserire un tag form permette di avere un markup più pertinente e di reagire al click di un bottone ma anche alla pressione del tasto invio.

Per controllare l'evento dell'invio di un form, possiamo usare `onSubmit` sul tag form!

```jsx
import { useState } from 'react'

function App() {

  const [firstName, setFirstName] = useState('Marco');


  const handleSubmit = () => {
   alert('FORM INVIATO!')
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={firstName}
        onChange={e => { setFirstName(e.target.value) }}
      <button>Invia</button>
    </form>
  )
}

export default App
```

Controlled Element

Evento onSubmit

Tuttavia, l'evento submit di un form, fa sì che la pagina venga ricaricata.

Per impedire questo, possiamo raccogliere l'**event** scatenatosi all'invio del form e usare il suo metodo `preventDefault()`!

Ora possiamo procedere con le nostre operazioni, ad esempio stampare il nome inserito nel form.

```jsx
import { useState } from 'react'

function App() {

  const [firstName, setFirstName] = useState('Marco');


  const handleSubmit = event => {
    event.preventDefault();
    console.log('Il nome inviato è:' + firstName)
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={firstName}
        onChange={e => { setFirstName(e.target.value) }}
      <button>Invia</button>
    </form>
  )
}

export default App
```